

第 5 章 NILM 推理服务模块设计实现

5.1 本章目标与模块边界

第 4 章已经说明了数据采集层如何将终端或模拟终端上报的二进制量测报文转换为结构化记录，并通过 JSONL 形式保留接收时间、设备编号、功率字段、解析状态和原始报文等信息。本章在此基础上进一步说明 NILM 推理服务模块的设计与实现。该模块的主要目标不是单独追求某个模型指标，而是把 NILM 模型从训练脚本转化为系统中可加载、可调用、可切换和可验证的推理能力。

本文系统按照“模型训练与导出、模型包管理、服务化推理、在线回放与集成验证”的路径组织推理服务模块。训练侧负责将公开数据集或结构化时序数据整理为模型输入窗口，并训练多设备功率预测模型；模型包侧负责把模型文件、归一化参数、后处理规则和接口说明打包成统一工件；服务侧负责加载模型包，对外提供离线区间推理和在线会话推理接口；回放与集成脚本负责用准备好的时间序列数据驱动服务运行，为后续系统测试和应用展示提供输入。

需要说明的是，本章重点描述推理服务模块的工程实现方式。模型识别误差、事件识别效果、接口测试和端到端运行结果将在第 7 章系统测试与结果分析中集中讨论。本章中出现的基准指标主要用于说明模型包注册和验收记录的组织方式，完整测试过程和结果讨论仍以后续测试章节为准。

5.2 数据处理与模型训练流程

NILM 推理服务的输入是总功率时间序列，输出是若干目标用电设备的功率估计值。当前 Stage-02 主线使用 1 秒采样粒度的数据，输入字段为 `mains_w`，目标设备字段为对应设备功率。根据项目记录，当前基准演示包面向 `kettle`、`microwave` 和 `toaster` 三类设备。对于第 4 章采集模块产生的三相功率记录，进入推理服务前可以按业务口径将 `powerA`、`powerB` 和 `powerC` 汇总为总功率字段；对于公开数据集回放，则直接读取已经处理好的总功率与设备功率列。

模型训练采用滑动窗口方式构造样本。设窗口长度为 `window_size`，窗口内连续总功率序列作为模型输入，窗口中心点对应时刻的各设备功率作为监督目标。窗口长度要求为奇数，这样可以明确中心点位置，避免输出时间戳与输入窗口之间出现半个采样周期的偏移。训练脚本首先按时间顺序划分训练集、验证集和测试集，然后对总功率和目标设备功率计算归一化参数，再将滑动窗口转换为模型可接收的张量形状。

模型结构采用 Seq2Point 风格的一维卷积网络。输入张量形状为 `[batch, window_size, 1]`，其中最后一维表示单通道总功率序列。模型内部先将输入转置为卷积层需要的 `[batch, channel, time]` 形式，再通过多层一维卷积提取时间局部特征，最后经全连接层输出多个设备的归一化功率值。多设备输出使同一模型可以同时给出多个目标设备的功率估计，服务层再按照模型元数据中的设备顺序将输出张量映射为业务字段。

训练完成后，模型不会直接以训练脚本中的中间状态交付给服务，而是导出为 ONNX 模型文件，并同时保存模型元数据、归一化参数和后处理规则。这样做的目的有两个。第一，服务运行时不依赖训练脚本内部对象，而通过稳定的模型包接口加载模型。第二，模型替换时只需要满足约定的模型包契约，服务代码不需要随每次训练结果同步修改。

表 5-1 给出了当前模型训练与导出流程中的主要数据对象。

对象	主要内容	作用
训练输入 CSV	timestamp_utc、mains_w、设备功率列	提供模型训练和离线/在线回放数据
滑动窗口样本	连续 window_size 个总功率点	作为模型输入
中心点目标	窗口中心时刻的多设备功率	作为监督学习目标
ONNX 模型	model.onnx	服务端推理运行文件
模型元数据	模型名、数据集、采样周期、窗口长度、设备列表、输入输出张量名	供运行时校验和字段映射
归一化参数	总功率均值方差、目标功率均值方差	供运行时前处理和反归一化
后处理规则	开关阈值、最短开关持续时间、可选线性校准参数	供事件抽取和输出修正

5.3 模型工件与版本管理

为了降低模型训练结果与服务代码之间的耦合，本文系统将可部署模型组织为标准模型包。模型包的接口版本为 `nilm_model_interface_v1`，一个完整模型包至少包含 `model.onnx`、`model_meta.json`、`normalization.json`、`postprocess.json`、`interface_spec.json` 和 `package_manifest.json` 六类文件。其中，`model.onnx` 保存模型结构和参数；`model_meta.json` 保存窗口长度、采样周期、设备列表、输入输出张量名称等元信息；`normalization.json` 保存前处理和反归一化所需参数；

`postprocess.json` 保存事件阈值和可选校准参数；`interface_spec.json` 记录模型包接口版本；`package_manifest.json` 记录包内文件大小和哈希值。

表 5-2 给出了模型包核心文件说明。

文件	内容	运行时用途
<code>model.onnx</code>	ONNX 格式模型	由 ONNX Runtime 执行推理
<code>model_meta.json</code>	模型名、数据集、采样周期、窗口长度、设备列表、张量名和张量形状	校验输入输出并映射设备顺序
<code>normalization.json</code>	输入总功率和目标设备功率的归一化统计量	对输入窗口归一化，对输出反归一化
<code>postprocess.json</code>	设备开关阈值、最短持续时间和可选线性校准参数	进行输出修正和事件抽取
<code>interface_spec.json</code>	接口版本和模型包说明	判断模型包是否兼容当前服务
<code>package_manifest.json</code>	包内文件路径、大小、SHA-256	防止文件缺失或包内容漂移

模型包导出由 `export_model_package.py` 完成。该脚本先检查训练工件目录中是否存在必需文件，再根据模型元数据和后处理规则生成接口说明文件，最后计算包内文件大小和 SHA-256 哈希并写入 `manifest`。服务加载模型包时，`ModelPackageRuntime.load` 会检查必需文件是否齐全、接口版本是否匹配，并在启用 `manifest` 校验时对文件大小和哈希进行核对。如果接口版本不一致、`manifest` 不匹配或文件缺失，加载过程会直接失败，从而避免服务在不确定模型状态下继续运行。

除单个模型包外，系统还通过注册表管理可选模型包。注册表记录当前 `active package` 路径、包 ID、`profile`、验收报告路径和聚合指标。根据项目记录，当前注册表包含 `event_optimized` 和 `error_optimized` 两类 `accepted package`，前者作为默认演示包，后者作为可回滚候选包。服务启动会优先读取注册表中的 `active package`；如需切换模型，只需通过管理脚本修改 `active package` 指针，再重新运行 `smoke` 或 `API` 检查即可。这种做法使模型替换从“修改服务代码”变为“切换模型包引用”，更符合工程系统的维护方式。

表 5-3 为当前注册表中的基准模型包记录。该表来自项目注册表和固定区间验收报告，用于说明版本管理方式；本机 CPU 环境下的模型包校验、服务 `API` 检查和在线回放结果将在第 7 章集中说明。

包 ID	类型	状态	MAE 平均值/W	RMSE 平均值/W	Event F1 平均值	说明
event optimized	事件优 化	accepted	11.2317	44.7314	1.0000	默认演 示包
error optimized	误差优 化	accepted	12.9034	62.7407	0.8333	回滚候 选包

5.4 离线推理服务设计

离线推理用于对指定时间区间内的总功率序列进行批量推理，并输出真值、预测值、误差指标和事件信息。该功能主要服务于模型效果检查、论文结果分析和应用层的区间展示。服务接口为 `/api/offline/infer`，请求中可以给定起止时间、事件匹配容忍秒数、批量推理大小和目标返回点数等参数。

离线推理的关键在于时间区间和模型窗口之间的对齐。由于 Seq2Point 模型需要使用中心点前后的上下文，服务不能只读取用户选择区间内的样本，而需要根据 `window_size` 向前和向后扩展一定时间范围。服务先加载包含扩展上下文的数据片段，使用滑动窗口构造批量输入，再对每个窗口执行模型推理。推理输出对应窗口中心点，因此服务会将预测结果与中心点时间戳重新对齐，随后再截取用户原始选择区间内的对齐结果。

完成对齐后，服务分别计算各设备预测功率与真值功率之间的误差，包括 MAE、RMSE、MeanDiff 和 MaxAbsDiff 等指标。同时，服务会基于后处理规则对真值序列和预测序列进行事件抽取，并按给定时间容忍范围匹配开关事件。为了避免前端或报告一次返回过多点，服务还会保留首尾点和事件点，并按目标点数对连续曲线进行降采样。这样既能展示总体趋势，又能尽量保留开关事件附近的关键变化。

离线推理输出可以分为四类。第一类是时间序列字段，包括时间戳、总功率、各设备真值功率和预测功率。第二类是误差指标，包括平均绝对误差、均方根误差、平均偏差和最大绝对误差。第三类是事件信息，包括真值事件、预测事件、事件数量和事件匹配情况。第四类是配置和上下文字段，包括模型版本、窗口长度、原始点数和降采样点数等。通过这些字段，离线推理接口不仅能返回一条预测曲线，也能为后续测试章节提供可量化的评价数据。

5.5 在线会话推理机制

在线会话推理用于模拟系统持续接收总功率采样点并滚动输出设备级预测结果。与离线区间推理不同，在线会话无法一次性读取完整时间段，而是随着样本逐条到达逐步更新内部状态。系统通过 `/session/start`、`/session/ingest`、`/session/latest` 和 `/session/reset` 组织在线推理生命周期。

会话启动时，服务根据请求参数或注册表 `active package` 加载模型包，并初始化在线会话运行对象。运行对象内部维护一个最大长度为 `window_size` 的总功率缓冲区，同时记录输入样本数量、预测次数、最近预测结果、预测历史和事件游标。每次调用 `/session/ingest` 时，服务将新到达的 `timestamp_utc` 和 `mains_w` 追加到缓冲区。如果缓冲区尚未达到窗口长度，服务返回 `ready=false`，并同时返回当前缓冲填充数量，表示模型尚未具备完整上下文。

当缓冲区长度达到 `window_size` 后，每个新样本都会形成一个完整输入窗口。服务将窗口转换为模型输入张量，调用模型运行时得到设备功率预测结果，再根据模型元数据中的设备顺序映射为 `pred_w` 字段。预测值会被追加到设备级预测历史中，随后运行时基于后处理阈值抽取开关事件。为了避免重复返回历史事件，会话对象为每个设备维护事件游标，只把本次推理中新出现的事件加入响应。

表 5-4 给出了在线会话最新状态的主要字段。

字段	含义
<code>ready</code>	当前缓冲区是否已经形成完整模型输入窗口
<code>timestamp_utc</code>	最近一次输入样本时间
<code>pred_w</code>	各目标设备当前预测功率
<code>events</code>	本次推理新增事件
<code>model_version</code>	当前会话加载的模型包文件名
<code>buffer_fill</code>	当前滚动缓冲区已填充样本数
<code>window_size</code>	模型要求的窗口长度
<code>ingest_count</code>	会话累计接收样本数
<code>pred_count</code>	会话累计完成预测次数

这种在线会话设计使服务能够在接口层体现实时推理过程中的状态变化。缓冲未满时，服务不会伪造预测结果；缓冲满后，服务按新样本持续输出预测和事件；需要重新开始回放或切换模型时，可以通过 `reset` 或重新 `start session` 清空状态。对于应用展示层而言，只需轮询 `latest` 状态或调用在线回放接口，就可以获得模型版本、最新预测、事件流和缓冲状态。

5.6 回放与集成验证机制

为了在缺少真实在线家庭总表输入的情况下验证推理服务链路，系统实现了 CSV 回放和集成脚本。回放器读取处理后的时间序列 CSV，按时间戳顺序将 `mains_w` 样本提交到 `/session/ingest`。回放速度可以通过倍速参数控制，既可以接近真实时间，也可以在演示和测试时加速运行。回放过程中，脚本会周期性查询 `/session/latest`，观察服务是否已经进入 `ready` 状态以及预测计数是否增长。

在服务内部，在线回放还可以通过 `/api/online/start`、`/api/online/stop` 和 `/api/online/status` 进行控制。`OnlineReplayController` 使用后台线程读取 CSV 数据，将样本持续写入当前会话，并在会话 `ready` 后记录真值功率和预测功率历史。状态接口返回当前回放是否运行、已发送行数、最近预测状态、历史曲线、在线误差指标以及真值事件流和预测事件流。该接口为前端展示和端到端测试提供了统一数据来源。

集成脚本则用于把模型包、服务和回放流程连接起来。`run_demo_smoke.py` 会启动服务，调用会话启动接口，然后运行回放脚本，最后检查 `/session/latest` 中的 `ready` 和 `pred_count` 字段。如果服务没有进入 `ready` 状态或没有产生预测结果，`smoke` 检查会失败。`run_service_api_checks.py` 进一步覆盖服务健康检查、离线推理接口、在线回放接口和停止接口，用于快速判断 Stage-02 服务链路是否具备演示和测试条件。

表 5-5 列出了推理服务相关接口。

接口	方法	作用
<code>/health</code>	GET	返回服务健康状态、接口版本和 <code>active package</code> 信息
<code>/session/start</code>	POST	加载模型包并启动在线推理会话
<code>/session/reset</code>	POST	清空当前在线会话缓冲和历史状态
<code>/session/ingest</code>	POST	向当前会话写入一个总功率采样点
<code>/session/latest</code>	GET	查询当前会话最新预测和事件状态
<code>/api/meta</code>	GET	返回模型、数据和预设区间元信息
<code>/api/offline/infer</code>	POST	对指定时间区间执行离线批量推理
<code>/api/online/start</code>	POST	启动服务端在线回放
<code>/api/online/stop</code>	POST	停止服务端在线回放
<code>/api/online/status</code>	GET	查询在线回放状态、历史曲线、指标和事件流

通过上述回放和集成机制，推理服务模块不再只是一个孤立的模型运行脚本，而是形成了可启动、可喂数、可查询、可切换模型包、可进行接口检查的服务链路。后续第 6 章可以基于这些接口说明应用展示层如何消费预测结果，第 7 章则可以基于 `smoke`、API 检查和固定区间推理进一步展开系统测试与结果分析。

5.7 本章小结

本章围绕 NILM 推理服务模块，说明了系统从时序数据处理、滑动窗口模型训练、模型包导出、运行时加载、离线区间推理、在线会话推理到回放集成的实现过程。模型训练侧采用多输出 Seq2Point 风格网络，将总功率窗口映射为多个目标设备功率；模型工程侧通过统一模型包契约保存 ONNX 模型、元数据、归一化参数、后处理规则和 manifest；服务侧通过 FastAPI 提供离线推理和在线会话接口，并在运行时对模型包接口版本和文件一致性进行校验。

与第 4 章的数据采集模块相比，本章的重点从“如何获得结构化量测记录”转向“如何将时序输入转化为可调用的设备级推理结果”。通过模型包注册表、active package 切换、在线滚动缓冲和回放脚本，系统将 NILM 模型封装为较稳定的服务能力，为后续应用展示和系统测试提供了基础。下一章将进一步说明应用界面如何调用推理服务接口，展示模型状态、功率曲线和事件结果。

第 7 章 系统测试与结果分析

7.1 测试目标与测试环境

前文已经分别说明了数据采集模块、NILM 推理服务模块和应用展示层的设计实现。本章从系统测试角度对主要功能链路和推理结果进行验证，重点回答三个问题：第一，数据采集模块能否在模拟终端输入下完成接收、解析和落盘；第二，训练机器导出的 NILM 模型包能否在本机 CPU 环境下被推理服务加载和调用；第三，离线推理、在线会话和在线回放是否能够形成可检查的输出结果。

本章测试采用模块测试与集成测试相结合的方式。数据采集模块测试复用第 4 章形成的 GUI 联调、JSONL 样本和压力联调证据；推理服务测试基于 Stage-02 模型包、处理后 CSV、模型验收报告和本机 CPU 运行输出；模型指标分析主要引用固定区间验收报告，并补充一次本机离线区间推理样例；在线回放测试则通过 smoke 脚本和服务端在线回放状态进行验证。

测试环境如表 7-1 所示。

表 7-1 测试环境说明

项目	内容
操作系统	Windows 本机环境
推理运行方式	ONNX Runtime CPU 推理
服务框架	FastAPI + Uvicorn
模型接口版本	nilm_model_interface_v1
主模型包	event optimized zip
回滚模型包	error optimized zip
推理数据	timeseries_1s_train_ready.csv
目标设备	kettle、microwave、toaster

需要说明的是，模型训练在具有 GPU 的训练机器上完成，本章验证的是训练结果在本机 CPU 环境下的加载、推理和服务调用能力。该安排符合系统部署场景：训练阶段可以依赖高性能计算资源，而推理服务需要具备在普通运行环境中加载模型包并输出结果的能力。

7.2 测试数据说明

Stage-02 推理测试使用处理后的 UK-DALE house_1 时间序列数据。数据质量报告显示，当前处理范围为 2013-03-17T19:12:50Z 至 2013-03-24T19:12:43Z，处理后的 train-ready 数据共 532616 行。数据字段包括 UTC 时间戳、总功率 mains_w，以及 kettle、microwave、toaster 三类目标设备的功率列。

表 7-2 给出推理数据基本情况。

表 7-2 推理数据说明

项目	内容
数据来源	UK-DALE house_1 处理后时间序列
采样粒度	1 秒
时间范围	2013-03-17 19:12:50 至 2013-03-24 19:12:43 (UTC)
train-ready 行数	532616
输入字段	timestamp_utc、mains_w
目标字段	kettle_w、microwave_w、toaster_w
处理规则	总功率不填补，设备功率使用最多 6 秒前向填充

该数据集主要用于模型训练、离线区间推理和在线回放验证。它并不等同于第 4 章采集后台生成的模拟终端数据。第 4 章采集模块验证的是终端上报数据进入系统的能力，而本章 Stage-02 推理测试验证的是 NILM 模型服务对标准时序输入的处理能力。两部分共同支撑系统从数据接入到模型推理的闭环。

7.3 数据采集模块测试

数据采集模块测试目标是验证采集后台在模拟终端输入条件下能否完成 TCP 接入、报文解析、设备状态更新和 JSONL 落盘。该部分测试基于第 4 章证据包，包含常规模拟联调和接收端压力联调两类。

常规模拟联调中，采集 App 监听 0.0.0.0:5000，模拟终端连接 127.0.0.1:5000，共启用 20 个设备。运行观察显示，20 个模拟终端均成功连接，设备状态、报文列表和实时趋势能够在界面中展示。最终落盘文件统计得到记录 320 条，其中成功解析记录 300 条，停止联调时产生连接异常记录 20 条。停止阶段产生的连接异常来自模拟终端主动结束连接，不影响对正常报文接收和解析能力的判断。

压力联调用于观察接收端在多连接和较高频率输入下的处理表现。压力发送脚本模拟 80 个连接，持续 20 秒，每个连接每 100 ms 发送一条 44 字节报文，理论输入速率约为 800 帧/秒。发送端实际发送 15920 条报文，连接错误为 0；接收端落盘记录 15920 条，成功解析记录 15920 条，状态分布为 Success=15920。压力联调结果如表 7-3 所示。

表 7-3 数据采集模块压力联调结果

指标	结果
模拟连接数	80
持续时间	20 秒
单连接发送间隔	100 ms

理论输入速率	约 800 帧/秒
发送端实际发送	15920 条
发送端连接错误	0
接收端落盘记录	15920 条
成功解析记录	15920 条
本机模拟成功解析比例	100%

从测试结果看，采集模块在本机模拟条件下能够完成定长报文接收、协议解析和结构化落盘，满足毕业设计原型阶段对数据接入链路的验证要求。不过，该测试仍属于本机模拟测试，不能替代真实硬件终端、跨主机网络和长期在线运行测试。因此，本文结论限定为“在本机模拟终端输入条件下采集链路可用”。

7.4 推理服务功能测试

推理服务功能测试用于验证模型包是否能被服务正确加载，以及服务接口是否能够完成模型信息查询、离线推理、在线回放和状态查询。测试前先使用模型包注册表确认 active package 是否可解析。命令如下：

```
python integration\manage_package_registry.py verify-active
```

输出显示 active package 指向 event optimized 模型包，解析路径为 stages/stage-02/model/artifacts/packages/kmt_multi_w301_20260323_155405_thopt_20260323_161139__event_optimized.zip，且 exists=True。这说明当前注册表与模型包文件位置一致，服务可以使用默认 active package 启动推理会话。

随后运行服务 API 检查：

```
python integration\run_service_api_checks.py
```

该脚本通过 FastAPI TestClient 检查

/health、/api/meta、/api/presets/offline、/api/presets/online、/api/offline/infer、/api/online/start、/api/online/status 和 /api/online/stop 等接口。运行结果输出 [api-check] PASS (session + offline + online)，说明推理服务的基本接口链路可用。

最后运行在线回放 smoke：

```
python integration\run_demo_smoke.py --data-csv
```

```
model\data\processed\house_1_1s_kmt\timeseries_1s_train_ready.csv --speed 200 --max-rows 320
```

smoke 脚本启动服务，加载 active package，然后将 CSV 中的总功率样本按加速倍率送入 /session/ingest。本次回放发送 320 行，模型窗口长度为 301，因此理论上在缓冲区填满后可以产生 20 次预测。实际输出显示 ready=True、buffer_fill=301、ingest_count=320、pred_count=20，与预期一致。

表 7-4 汇总了推理服务功能测试结果。

表 7-4 推理服务功能测试结果

测试项	预期结果	实际结果
active package 校验	模型包路径可解析且文件存在	exists=True
API 检查	session、offline、online 接口通过	[api-check] PASS
在线 smoke	回放后进入 ready 状态并产生预测	ready=True, pred_count=20
模型版本	latest 状态返回当前模型包名	返回 event optimized zip 文件名

上述结果说明，GPU 训练机器导出的 ONNX 模型包可以在本机 CPU 环境下由 Stage-02 推理服务加载，服务接口能够完成离线和在线两种调用路径。

7.5 模型识别指标分析

模型识别效果主要通过固定区间验收报告进行分析。当前注册表中包含两个 accepted 模型包：event optimized 作为默认演示包，error optimized 作为回滚候选包。两者均使用相同的目标设备和接口版本。固定区间验收聚合指标如表 7-5 所示。

表 7-5 固定区间模型验收聚合指标

模型包	MAE 平均值/W	RMSE 平均值/W	Event F1 平均值
event optimized	11.2317	44.7314	1.0000
error optimized	12.9034	62.7407	0.8333

从表 7-5 可以看出，event optimized 模型包在固定验收区间上的 MAE、RMSE 和 Event F1 均优于 error optimized 包，因此本文后续演示和测试默认使用 event optimized 作为 active package。error optimized 仍可作为回滚候选包保留，其意义在于验证模型包注册表可以管理多个候选模型，并在需要时通过 active package 切换实现回滚。

为了补充固定验收报告之外的本机推理证据，本章还选取 2013-03-19T17:35:02Z 至 2013-03-19T18:42:14Z 作为离线推理样例区间，通过 /api/offline/infer 获取服务返回的指标和事件结果。样例结果如表 7-6 所示。

表 7-6 本机离线推理样例结果

指标	结果
MAE 平均值/W	17.5778
RMSE 平均值/W	112.3741
MeanDiff 平均值/W	7.2924
MaxAbsDiff/W	2481.4275
Event F1 平均值	1.0000
原始对齐点数	3571
返回点数	420

该样例区间中，三个目标设备的事件 F1 均为 1.0000，说明在 10 秒事件匹配容忍范围内，服务能够正确匹配该区间内的开关事件。与此同时，MaxAbsDiff 达到 2481.4275 W，说明在某些高功率短时变化点上，预测值与真值之间存在较大瞬时偏差。因此，评价 NILM 推理效果时不能只依赖单一指标。MAE 和 RMSE 反映整体功率误差，Event F1 反映开关事件识别能力，MaxAbsDiff 则提示高功率瞬时误差风险。

7.6 在线回放测试

在线回放测试用于验证推理服务能否在连续输入样本的情况下维护滚动窗口并持续输出预测结果。除 smoke 脚本外，本章还通过服务端在线回放接口执行了一次结构化样例测试。测试区间为 2013-03-19T17:24:44Z 至 2013-03-19T17:34:44Z，回放速度为 500 倍，最大发送行数为 420。

在线状态返回结果显示，回放结束时 sent_rows=420, running=false, latest 状态中 ready=true, ingest_count=420, pred_count=120。由于模型窗口长度为 301，回放 420 行后可以形成 120 个完整窗口预测，该结果与窗口机制一致。在线状态还返回了历史真值和预测曲线，三个设备的 true_w 和 pred_w 均包含 120 个点，说明在线回放接口不仅能输出最新预测，也能为前端趋势展示提供连续历史序列。

表 7-7 给出在线回放样例结果。

表 7-7 在线回放样例结果

字段	结果
回放发送行数	420
模型窗口长度	301
累计预测次数	120
latest ready	true
历史时间点数	120
kettle 真值/预测点数	120 / 120
microwave 真值/预测点数	120 / 120
toaster 真值/预测点数	120 / 120

该测试说明，在线会话缓冲区、滚动窗口推理、latest 状态和历史曲线缓存能够协同工作。对于应用展示层而言，前端可以通过 /api/online/status 获取最新状态、历史曲线和事件流，从而实现在线模拟展示。

7.7 测试结果讨论

综合上述测试，本文系统的主要链路已经形成可运行证据。数据采集模块能够在本机模拟终端输入条件下完成 TCP 接入、协议解析和 JSONL 落盘；NILM 推理服务能够加载 GPU 训练机器导出的标准模型包，并在本机 CPU 环境下完成离线区间推理和在线

会话推理；回放脚本和服务端在线回放接口能够持续喂入总功率样本，并返回模型版本、预测值、事件和历史曲线。

从模型效果看，event optimized 模型包在固定区间验收中取得了较好的事件识别结果，Event F1 平均值为 1.0000，适合作为系统演示和论文测试的主模型包。误差指标方面，MAE 和 RMSE 可以说明总体功率预测误差，但离线样例中的 MaxAbsDiff 表明高功率短时事件仍可能产生较大瞬时偏差。因此，本文在结果分析中同时保留功率误差指标和事件识别指标，而不只用单一平均误差评价系统效果。

系统仍存在若干限制。首先，采集模块测试主要基于本机模拟终端，尚未覆盖真实硬件终端、跨主机网络和长期连续运行场景。其次，推理服务测试使用公开数据集处理后的标准 CSV 回放，尚未直接使用真实家庭现场数据验证模型泛化能力。最后，当前应用展示层的测试证据还需要结合第 6 章界面截图进一步补充。后续工作可以在真实终端接入、长时间运行监控、更多家庭数据集和应用层端到端截图方面继续完善。

7.8 本章小结

本章从数据采集、推理服务、模型指标和在线回放四个方面对系统进行了测试与分析。测试结果表明，数据采集模块在本机模拟条件下可以稳定完成报文接收、解析和落盘；Stage-02 推理服务能够在 CPU 环境下加载标准模型包，并通过离线推理和在线回放接口输出设备级预测结果；固定区间验收和本机离线样例为模型识别效果提供了量化依据。整体来看，系统已经具备从数据输入、模型推理到结果输出的基本闭环能力，满足毕业设计原型系统的验证目标。